ML Assignment2

UNIVERSITY OF HARTFORD

K-Nearest Neighbors

Classification and Model Evaluation

Building a K-Nearest Neighbors classifier to predict mobile phone price ranges.

Student: John Bennett

Course: CS 569 Machine Learning

Fall 2025

September 30, 2025

Contents

1 Objective **1**

2 Data Set Overview and Cleaning **1**

3 Data Preparation **3**

4 Model Training **4**

5 Results and Discussion **5**

6 Conclusion **7**

1 Objective

In this assignment, I used a real-world dataset to build a K-Nearest Neighbor classifier to predict mobile phone price ranges in order to practice:

- Training a machine learning model.
- Evaluating its performance using accuracy, precision, recall, and F1-score.
- Experiment with different values of k and analyze their effects on model performance (along with accuracy plot for k-candidate range).

2 Data Set Overview and Cleaning

Table 1: Summary Statistics of Mobile Attributes

attribute	mean	std	min	25%	50%	75%	max	data type	number null	number unique
battery power	1238.518500	439.418206	501.000000	851.750000	1226.000000	1615.250000	1998.000000	int64	0	1094
blue	0.495000	0.500100	0.000000	0.000000	0.000000	1.000000	1.000000	int64	0	2
clock speed	1.522250	0.816004	0.500000	0.700000	1.500000	2.200000	3.000000	float64	0	26
dual sim	0.509500	0.500035	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
fc	4.309500	4.341444	0.000000	1.000000	3.000000	7.000000	19.000000	int64	0	20
four g	0.521500	0.499662	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
int memory	32.046500	18.145715	2.000000	16.000000	32.000000	48.000000	64.000000	int64	0	63
m dep	0.501750	0.288416	0.100000	0.200000	0.500000	0.800000	1.000000	float64	0	10
mobile wt	140.249000	35.399655	80.000000	109.000000	141.000000	170.000000	200.000000	int64	0	121
n cores	4.520500	2.287837	1.000000	3.000000	4.000000	7.000000	8.000000	int64	0	8
pc	9.916500	6.064315	0.000000	5.000000	10.000000	15.000000	20.000000	int64	0	21
px height	645.108000	443.780811	0.000000	282.750000	564.000000	947.250000	1960.000000	int64	0	1137
px width	1251.515500	432.199447	500.000000	874.750000	1247.000000	1633.000000	1998.000000	int64	0	1109
ram	2124.213000	1084.732044	256.000000	1207.500000	2146.500000	3064.500000	3998.000000	int64	0	1562
sc h	12.306500	4.213245	5.000000	9.000000	12.000000	16.000000	19.000000	int64	0	15
sc w	5.767000	4.356398	0.000000	2.000000	5.000000	9.000000	18.000000	int64	0	19
talk time	11.011000	5.463955	2.000000	6.000000	11.000000	16.000000	20.000000	int64	0	19
three g	0.761500	0.426273	0.000000	1.000000	1.000000	1.000000	1.000000	int64	0	2
touch$_{s}creen$	0.503000	0.500116	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
wifi	0.507000	0.500076	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
price range	1.500000	1.118314	0.000000	0.750000	1.500000	2.250000	3.000000	int64	0	4

The data set overall has 2000 rows with 21 columns with aggregate information Provided in Table 1.

Table 1 provides an overview of the contents of the dataset in its original form. Each attribute name is listed in the first column with summative statistics (including the mean, standard deviation, minimum value, quartile distribution, and maximum) of each attribute's values in successive columns, followed by the attribute's data type, number of null values, and number of unique values for each attribute in the remaining columns. This provides some inferences about each of the attributes and the role the will play in the training of the K-Nearest Neighbors Model. The meaning of each attribute is as follows and can be verified at Kaggle:

battery_power Battery capacity of the mobile phone in mAh (milliampere-hour).

blue Indicates the presence of Bluetooth (`1` = yes, `0` = no).

clock_speed Processor speed in GHz. Higher values usually mean faster processing.

dual_sim Indicates whether the phone supports dual SIM cards (`1` = yes, `0` = no).

fc Front camera resolution in megapixels (MP). Higher values indicate better front camera quality.

four_g Indicates 4G network support (`1` = yes, `0` = no).

int_memory Internal memory (ROM) in GB. Higher values mean more storage space for apps, files, and media.

m_dep Mobile depth (thickness) in cm. Physically measures how thick the phone is.

mobile_wt Weight of the mobile phone in grams (g).

n_cores Number of processor cores. More cores can improve multitasking and performance.

pc Primary camera resolution in megapixels (MP). Higher values usually indicate better image quality.

px_height Pixel resolution along the height of the screen. Used with `px_width` to determine screen resolution.

px_width Pixel resolution along the width of the screen.

ram RAM size in MB. More RAM allows more applications to run simultaneously and improves performance.

sc_h Screen height in cm. Physical dimension of the display.

sc_w Screen width in cm. Physical dimension of the display.

talk_time Maximum talk time of the battery in hours.

three_g Indicates 3G network support (`1` = yes, `0` = no).

touch_screen Indicates if the phone has a touchscreen (`1` = yes, `0` = no).

wifi Indicates the presence of Wi-Fi support (`1` = yes, `0` = no).

price_range Target variable indicating the price category of the phone:

- `0` = low cost
- `1` = medium cost
- `2` = high cost
- `3` = very high cost

The table shows that a minimum amount of cleaning is necessary while giving clues to which attributes with numeric values are actually categorical data. There are no nulls in the data, meaning that either there are no unknown values or those values are coded in a different way. Of the attributes with low numbers of distinct values, the attributes with minimum/maximum values of 0/1, respectively, along with having only two unique values indicates that these attributes are actually binary categorical attributes, identifying the dataset's records having or lacking the feature indicated by the attribute. It must be assumed that these attributes (blue, dual sim, four g, three g, touch screen, and wifi) all have zero values that have meaning and are not indicators of unknown/incomplete information. Having eight unique values for the the number of cores logically indicates that values refer to a core count rather than a category but that it is a discrete value where the mean and standard deviation are less meaningful than the quartile values. The exception is the fact that the relatively close values of the mean and 50 percentile (median) indicate that the distribution of this attribute is relatively normal or uniform, suggesting a lack of outliers. Price range consists of four values indicating that this attribute is categorical and can be used as classes

for each record. Having four distinct values suggests that zero is a class and not a placeholder.

Of the attributes with medium and highly distinct sets of values, several have minimum values of zero where it they are not consistent with what is being measured, indicating that they are not real values and are instead place holders. For example, the screen width (sc w) indicates the width of the screen so the zero values for this attribute have no meaning as there can't be a phone with no screen width. Attributes with this characteristic include front camera (fc), whose values are megapixels, pixel resolution for the height (px height), and screen width (sc w). Each of these were further evaluated for Null values where zero was replaced with NaN (PANDAS library's default numeric null type) and the overview table was re-evaluated.

The results of accurately updating zero values for the aforementioned attributes shows that front camera (fc) has 474 NaNs (24%), screen width (sc w) has 180 NaNs (9%), and pixel resolution for height (px height) has 2 NaNs (0.1%). In all cases the percentage is low enough to keep each attribute and instead impute values. The technique used for this is dot-decking using impute k-nearest neighbors from Scikit-Learn.

3 Data Preparation

K-Nearest Neighbors is a supervised machine learning algorithm that uses the data objects within its training dataset to classify unseen data object based on their similarity to one or more of the objects in the training data set. The model is different from other machine learning algorithms in the sense that it doesn't actually generate a model of patterns within the training dataset globally, but instead relies on the training data object's known classification values (class feature) and the distribution of the other feature values (feature tuple) for comparison with those same feature values in not yet classified data objects. The algorithm does this by establishing a feature space that is typically a high-dimensional euclidean space where each non-class feature accounts for one of the dimensions in this space. The algorithm then determines the distances between each unclassified data object's feature tuple and the feature tuples of the objects within the training data so that the k nearest training objects can be identified as those with the k shortest distances from the unclassified data object. It then assigns the classification most common within the set of k-nearest neighbors to the data object being classified. Said differently, these steps effectively define a hyper-dimensional spherical subspace that is centered at the current unclassified data object and uses the mode of the training object's class frequency distribution within this subspace as the class of the unclassified data object.

There are two main implications of the algorithmic steps. Because the model is essentially the implicit vector space of the training data itself, the implied requirement is that all of the data training feature tuples must contain numeric elements meaning that any non-numeric features that contribute to the dimensions of the feature tuples must be converted to numeric representations of its values. The second implication is that the value of k can have a massive impact on the classification assigned to each unclassified data object where a classification can change based on the number of neighbors considered. This second implication, however, is addressed in the training of the model itself. Therefore, the first implication is the primary concern in preparing the data. In this particular case all non-numeric attributes (situationally, those that indicate binary states) already have numeric values, so this implication is inherently addressed in the source data.

Pertaining to the values of the feature tuples, normalization is typically performed on

the values of both the training data and the unseen data in order to improve classification performance. Because the vector space is not inherently aware of variation in units used in the dimensions, the value ranges of dimensions that are large, relative to the ranges of other dimensions, will lead to those dimensions dominating the classification of the data since it diminishes the variation that exists in the feature space overall for dimensions with small ranges (such as the binary features). By normalizing each feature distribution to its respective z-scores, the feature values are all scaled so that they have a mean of zero and are relative to a standard deviation of one. (Effectively, the original values are transformed to the number of standard deviations their initial value is from their original mean for there respective feature distribution). Because these z-scores are on the same scale, each feature can have a consistent impact of the classification overall.

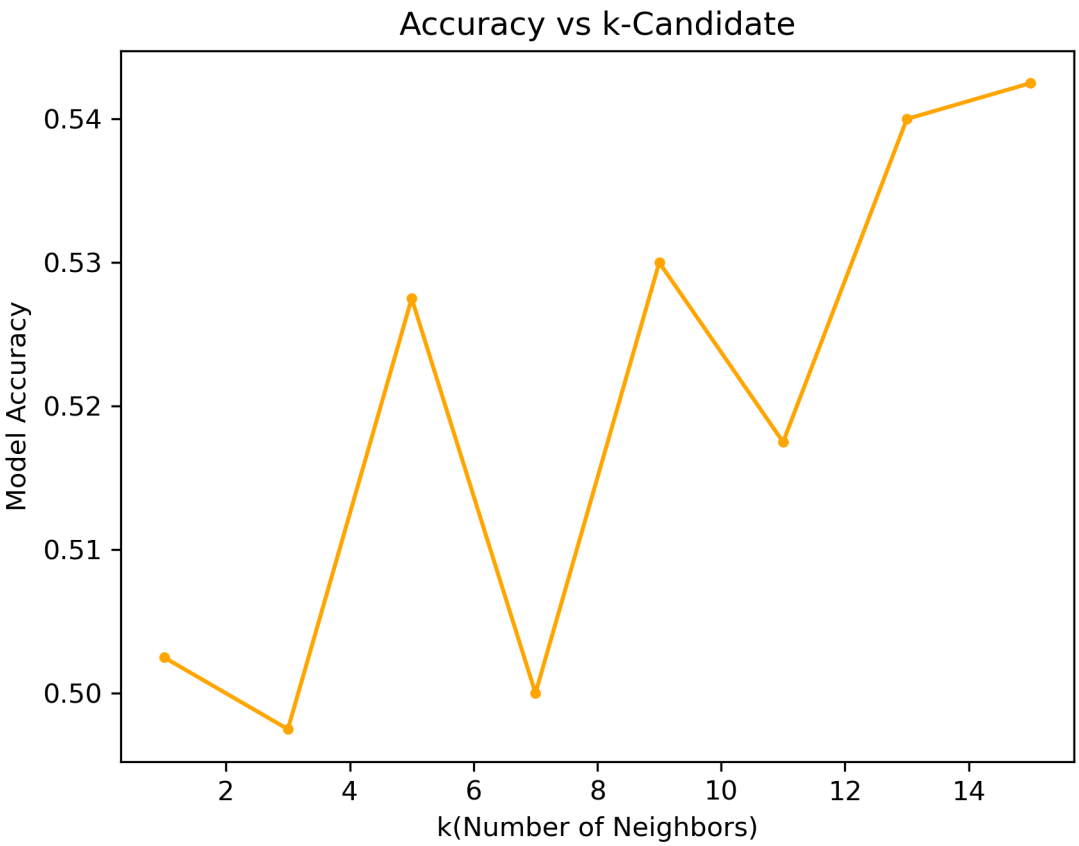
To prepare the data with respect to these implications and considerations, the attribute that contains the classes being trained for needs to be distinguished from the attributes the data is trained on for the sake of assessing performance measures. The data was then further subset to establish training and testing datasets. This was achieved using train_test_split using Scikit-learn's model_selection module. The training data was set to 80% of the data and the testing set 20% of the data resulting in the former having 1600 data objects and the latter having 400. The data sets containing the training and testing feature tuples were then normalized to z-scores using the StandardScalar class of Scikit-learn's preprocessing module.

4 Model Training

As mentioned the number of neighbors considered for each classification is an impactful parameter that is chosen for the model during its generation. In the context of large data, predetermining what the best k-value might be is difficult. As such, it is common to provide a list of candidate k-values that are then each tested for performance. In this case, it will be tested based on the accuracy that each k-value produced when performance was considered relative to the testing set of feature tuples. The model chosen then, is the one with the best overall performance with respect to accuracy. Here this was performed relative to the testing set but it is common to choose k by validation techniques on the training set itself and clarify performance on the test set independent of finding the k-value to avoid the test set playing a role in defining the model by influencing the k-value chosen.

The models were trained using the KNeighborsClassifier class from Scikit-learn's neighbors module. The tested k-values included the set of odd numbers from 1-15 and performance measures were collected from each model generated using these k-values. The performance measures were then used to determine the model that performed the best based on the clarifications made in the results and discussion. These measures were created for each model so that both class specific measures and aggregate measures for each model could be assessed for situational analysis of the best model.

5 Results and Discussion



The above graph shows the overall accuracy for the models generated for each k-candidate value. It shows that consideration of the mode classification of the nearest 15 neighbors produces the most accurate model. In some cases when using the test set to validate a model, the highest score can indicate overfitting but the overall accuracy is not very high so concerns about overfitting are likely not relevant. With this in mind, a k-value of 15 is the best model if you are only concerned about accuracy for the model across all classes.

Table 2: Aggregate Performance Table

k_candidates	Accuracy	Precision	Recall	F1
1	0.5025	0.513412	0.496990	0.501622
3	0.4975	0.520916	0.487339	0.489456
5	0.5275	0.556939	0.520710	0.528760
7	0.5000	0.528468	0.493224	0.500187
9	0.5300	0.539289	0.519706	0.523173
11	0.5175	0.527780	0.507009	0.509919
13	0.5400	0.561621	0.532805	0.532942
15	0.5425	0.568458	0.536818	0.540536

The table above provides more detail about how each k-candidate model performed. It includes the aggregate precision and recall, as well as the F1 score derived from these measures. The precision gives you a sense of how reliable the individuals models, which is

to say how often a prediction for a specific class is accurate across all of the classes. The recall (or sensitivity) provides the proportion of the objects of a class in the test data that are predicted correctly across all of the classes indicating which models are best for getting to the right classification without regards for the instances where the prediction of not belonging to a specific class is not accurate. The F1-score provides a perspective on the balance of these two measures in each model. When looking for the model with the best balance between sensitivity and prediction, you use the F1-score. This balance provides the means to determine the model that is best at correctly predicting a specific class generally while also accounting for how reliable predictions for that class are. When considering these measures in this particular context however, there is strong correlation between these measures where generally, the value of precision, recall, and F1-score all increase as accuracy increases. There are exceptions as seen in the increase in precision for k = 3 vs k = 1 where the accuracy decreases. However, the general increase in scores when comparing the models is a strong enough trend to rely on accuracy and further consider only the 2 best performing models which were for k = 13, and k = 15.

It is important to note that the performance of the model overall is pretty low. Given the baseline of k = 1, all of the performance scores are around 50% and when you compare that to k = 15, there is not much improvement. This indicates that the model is about as likely to predict the correct class as not. In addition, the relatively low increase in scores between considering one neighbor versus 15 neighbors indicates that the data does not provide the inherent meas to improve prediction that much. Notwithstanding this, there are instances where focusing one one measure or another might make the models more applicable. For example there are instances where correct prediction for a specific class is more important than overall accuracy of the model. In this context if the goal is to validate the determination of what value ranges for different features are more important for increasing or decreasing profit for a manufacturer, a model with high precision for high and low cost mobile devices can be more reliable than a model that is accurate overall. Where there is no concern for middle priced items because premium is referred, it is better to focus on what to include and not exclude in a mobile model. With this in mind precision can provide a sense of reliability for the model with respect to specific classes providing how often a prediction for a specific class is accurate. Additionally, the recall can indicate how good a model is at getting the prediction for a specific class correct. Considering this instance, the model does show meaningful performance.

Table 3: 13 Neighbors Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
0	0.67	0.81	0.73	105
1	0.36	0.42	0.39	91
2	0.37	0.41	0.39	92
3	0.85	0.49	0.62	112
Accuracy		0.54		400
Macro Avg	0.56	0.53	0.53	400
Weighted Avg	0.58	0.54	0.54	400

15 Neighbors Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.66	0.73	0.69	105
1	0.38	0.44	0.41	91
2	0.38	0.46	0.42	92
3	0.85	0.52	0.64	112
Accuracy		0.54		400
Macro Avg	0.57	0.54	0.54	400
Weighted Avg	0.59	0.54	0.55	400

For the existing data attributes in the training data, the ability of all of the models to predict of low (class 0) and high cost (class 3) mobile devices is relatively high. This is a trend for most of the models, but since the highest average F1-score for these two classes is seen in the k = 13 model, this model is situationally better than the k=15 while have almost exactly the same overall accuracy. Therefore, the k=13 model will likely offer the best analytical value for studying what ranges of different features most effects price of mobile devices for the lowest and highest price classes.

6 Conclusion

Of the range of the models created using K-Nearest Neighbors, none performed particularly well overall. There may be some applications for the better performing models but it is likely that building models with different algorithms will yield better classification results and provide more reliable means of analysis. A similar analysis was performed for the same dataset that underwent the same cleaning process using Random Forest, and the models performed much better overall.